Parameterized Matching with Mismatches *

Alberto Apostolico
Department of Computer Sciences, Purdue University,
West Lafayette, IN 47907, USA *and*
Dipartimento di Ingegneria dell' Informazione,
Università di Padova, Padova, Italy
axa@cs.purdue.edu †

Péter L. Erdős
A. Rényi Institute of Mathematics
Hungarian Academy of Sciences
Budapest, P.O. Box 127, H-1364 Hungary
elp@renyi.hu ‡

Moshe Lewenstein
Department of Computer Science
Bar-Ilan University
Ramat Gan 52900, Israel
moshe@cs.biu.ac.il

Abstract

The problem of approximate parameterized string searching consists of finding, for a given text $x = x_1x_2...x_n$ and pattern $y = y_1y_2...y_m$ over respective alphabets Σ_t and Σ_p, the injection π_i from Σ_p to Σ_t maximizing the number of matches between $\pi_i(y)$ and $x_ix_{i+1}...x_{i+m-1}$ $(i = 1, 2, ...n - m + 1)$. We examine the special case where both strings are run-length encoded, and further restrict to the case where one of the alphabets is binary. For this case, we give a construction working in time $O(n + (r_p \times r_t)\ \alpha(r_t)\ \log(r_t))$, where r_p and r_t denote the number of runs in the corresponding encodings for y and x, respectively, and α is the inverse of the Ackermann's function.

1 Introduction

String searching is one of the basic primitives of computation. In the standard formulation of the problem, we are given a pattern and a text and it is required to find all occurrences of the pattern in the text. Several variants of the problem have also been considered, e.g., allowing mismatches, insertions, deletions, swaps etc. In the parameterized variant, a match exists at one position of the text if the alphabets of pattern and text can be consistently mapped into one another in such a way that all characters match pairwise. This variant was introduced and studied by B. Baker [2, 3] and others, motivated by issues of program

*Work performed in part while on leave at ZIF, University of Bielefeld, Germany, within framework of the Special Year on "General Theory of Information Transfer and Combinatorics".

†Supported in part by an IBM Faculty Partnership Award, by the Italian Ministry of University and Research under the National Projects "Bioinformatics and Genomics Research" and FIRB RBNE01KNFP, and by the Research Program of the University of Padova.

‡This work was supported in part by the Hungarian NSF, under contract Nos. T37846, T34702.

compaction in software engineering. In this paper, we study approximate variants of the problem where a (possibly controlled) number of mismatches is allowed.

Specifically, we seek to find, for given text $x = x_1x_2...x_n$ and pattern $y = y_1y_2...y_m$ over respective alphabets Σ_t and Σ_p, the injection π_i from Σ_p to Σ_t maximizing the number of matches between $\pi_i(y)$ and $x_ix_{i+1}...x_{i+m-1}$ $(i = 1, 2, ...n - m + 1)$. We examine the special case where both strings are run-length encoded, and further restrict to the case where one of the alphabets is binary. For this case, we give a construction working in time $O(r_p \times r_t)$, where r_p and r_t denote the number of runs in the corresponding encodings for y and x, respectively.

This paper is organized as follows. In the next section, we give some basic definitions and properties, and derive the combinatorial facts used in our construction. Section 3 is devoted to the design and description of our algorithm. The last section lists conclusions and plans of future work.

2 Properties

Given a *string* $s = s_1s_2...s_n$ of *length* $|s| = n$ over an alphabet Σ_1, and an injection π from Σ_1 to some other alphabet Σ_2, the *image* of s under π is the string $s' = \pi(s) = \pi(s_1)o...o\pi(s_n)$, that is obtained by applying π to all characters of s.

Given two strings w and u of the same length over alphabets Σ_1 and Σ_2 and an injection π from Σ_1 to Σ_2 the π-*match* between w and u is the number of matches between $\pi(w)$ and u. The problem of *approximate parameterized matching* between w and u consists of finding a π of maximum π-match. For given input text x $(|x| = n)$ and pattern y $(|y| = m)$, the problem of *approximate parameterized searching (APS)* for y in x consists of computing the *approximate parameterized matching* between y and every (consecutive) substring of length m of x. Hence, approximate parameterized searching requires computing the π yielding maximum π-match for y at each position of x. Of course, the best π is not necessarily the same at every position.

The general problem of APS can be solved in $O(nm(\sqrt{m} + \log n))$ by reduction to bipartite graph matching (see, e.g., [6, 8]): each relative alignment defines one such graph in which edges are weighted according to the number of matches induced by each character "hits", and the problem is to choose a set of edges of maximum weight.

For fixed sized text and pattern alphabets there are a constant number of possible injections. Hence we can apply the convolution method [5] to try them out individually and then compare them point by point, in time $O(n \log n)$ overall. When $|\Sigma_p| = 1$, the problem has a trivial linear solution: slide over the text with a window of size equal to the pattern length m while keeping track of which text character scores the maximum number of matches. Following initialization at the first position, it is enough at every step to update the number of matches to check the text character leaving to face the pattern and the text character entering to face the pattern.

Such simplifications appear to fade as soon as either the text or the pattern alphabet

alone can be arbitrary while the other is binary.

In this paper, we concentrate on special cases of APS where the assumption is made that x and y are presented in their respective run-length encodings, denoted $X = X_1X_2 \dots X_{r_t}$ and $Y = Y_1Y_2 \dots Y_{r_p}$, respectively. The generic run, say $X_k = x_i x_{i+1} \dots x_{i+l-1}$, corresponds to a maximal substring of consecutive occurrences of the same symbol, and it has an *interval* $l_k = [i, i+l-1] = [L_{l_k}, R_{l_k}]$ naturally associated with it. For ease of notation we use the l_k's to denote both the intervals and their respective lengths. Thus, the list of integers $l_1, l_2 \dots l_{r_t}$ specifies the run intervals for X. Clearly, knowing the first symbols and the run intervals for a binary string X entirely specifies that string. Moreover, note that the interval lengths can be derived from L_{l_k}, the left-end of the interval, and from $L_{l_{k+1}}$, the left-end of the following interval. Hence, another way of specifying the run intervals is simply by the *left-end list*, $L_{l_1}, L_{l_{r_2}}, ..., L_{l_{r_p+1}}$ (or, $..., L_{l_{r_t+1}}$) , where $L_{l_{r_p+1}} = |P| + 1$ (or, $L_{l_{r_t+1}} = |T| + 1$, respectively) is added for the sake of consistent computation. We also use Σ_t and Σ_p to denote the text and pattern alphabet, respectively.

3 Run-Length Algorithm for Binary Alphabets

Our goal is to do APS in time $O(r_p \times r_t)$, but we must be prepared to pay an additive term linear in n, the length of x, and perhaps some logarithmic overhead. In practice, one may expect less than one run of pattern and text to end up juxtaposed at any position of the pattern, whence $O(r_p \times r_t)$ would be sub-linear.

We consider first the easy variant where $|\Sigma_p| = |\Sigma_t| = 2$. Note that $O(n \times r_p)$ or $O(m \times r_t)$ is doable in this case. For example, initialize the matching corresponding to aligning the pattern beginning in the first position of the text. Clearly, the π-mismatch at this position results from trying out the two possible assignments for the zeroes and ones and choosing the best one. Next, following every unit shift of the pattern, for each one of the two competing injections scan the runs of the pattern and update the matches for each run. We show next that this scheme can be extended to run in time $O(r_p \times r_t)$. Towards this, we need to introduce additional notions.

Consider the left-end list of the text $L_{l_1}, ..., L_{l_{r_t+1}}$. When deciding on the APS of the pattern at the specific length-m substring beginning at location i of T one desires to align the pattern with the appropriate substring. It is convenient to view this alignment as a shift of the text $i-1$ positions to the left. The *i-left-end list* is the list $L_{l_1} - (i-1), L_{l_{r_2}} - (i-1), ..., L_{l_{r_t+1}} - (i-1)$. However, since this list is defined for the pattern, the i-left-end list will only contain the elements that are positive and no more than $|P| = m$.

Definition 1 *(Fusion) Given the sequence of run intervals of text and pattern, the* i-fusion *is the sequence of intervals defined by the left-end list resulting from the merge of the left-end list of the pattern runs with the i-left-end list of the text runs. We call an arbitrary i-fusion a* fusion.

The example, depicted in Figure 1, illustrates the fusion of the pattern runs with the text runs at location 22 of the text, i.e. a 22-fusion.

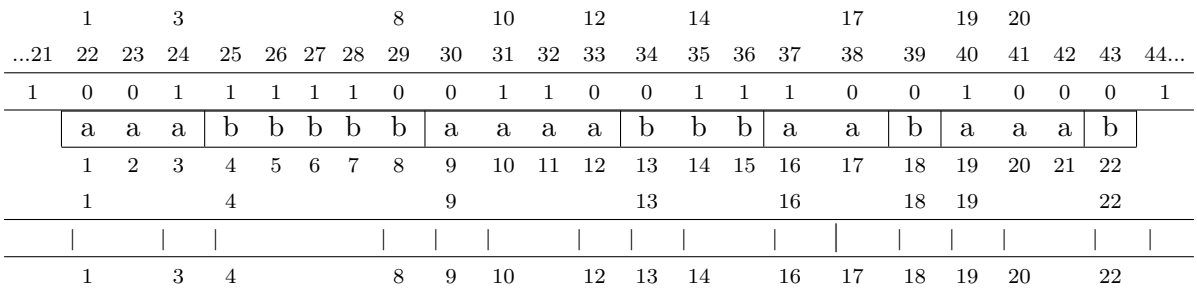

Figure 1: Illustrating the 22-fusion of pattern and text intervals. The lines contain: the 22-left-end list, text locations, text, pattern, pattern locations, left-end pattern list, depicted 22-fusion, left-end list of 22-fusion.

Each left-end list of an i-fusion, $\mathcal{L} = (L_1, L_2, \ldots, L_p)$, contains elements from the left-end list of the pattern and elements of the i-left-end list of the text. An element L_i is said to be either a pattern element or a text element according to its origin. An element can be both.

Definition 2 *(Neighbor) Let $\mathcal{L} = (L_1, L_2, \ldots, L_p)$ be a left-end list of a fusion. The* neighbor *of each text element L_i is $max_{j \le i}\{L_j \mid L_j$ is a pattern element$\}$ and the* neighbor distance *is the distance between them, i.e. $L_i - max_{j \le i}\{L_j \mid L_j$ is a pattern element$\}$.*
(Bump distance) The bump distance *is the minimum neighbor distance over all text elements L_i. We say that there is a* bump *if the bump distance is 0, i.e. if there is an element that is both a pattern and text element.*

During the process of finding the APS for each text location we will be sliding the pattern over the text, or alternatively sliding the text to the left. The fusion undergoes changes at every shift of the pattern against the text, changing from an i-fusion to an $i+1$-fusion with a unit shift. After a number of shifts equal to the bump distance a *bump* takes place. Clearly, exactly $r_p \times r_t$ bumps occur throughout our matching process.

We focus now on a single run of the pattern (see Fig. 2) and consider the changes in its contribution to the matching under unit shifts. Let l be (the length of) this run, and $x_i \ldots x_{i+l}$ the substring of text facing it.

Fact 1 *The update of the contribution of the run to the match depends only on the values of x_i and x_{i+l}. Specifically, this contribution is not affected if $x_i = x_{i+l}$ and it is either increased or decreased by 1 if $x_i \neq x_{i+l}$.*

Proof: The case $x_i = x_{i+l}$ does not require explanation. If $x_i \neq x_{i+l}$ and x_i matches the character of the run, then the number of the matches in the run decreases by 1. Otherwise, the number of the matches increases by 1. ∎

Fact 1 supports a simple algorithm for this instance of APS. The algorithm works as follows.

1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
|a	a	a	b	b	b	b	b	a	a	a	a	b	b	b	a	a	b	a	a	a	b |
32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53

Figure 2: Segment of a binary pattern facing one run in a binary text

We assign two counters to the task of keeping track of the π-match under either bijection $\pi : \Sigma_p \to \Sigma_t$. Specifically, c will tally the π-match under the *identity* $\pi(0_p) = 0_t$, and $\bar{c}$ will do the same for the *switch* $\pi(0_p) = 1_t$, with transparent notation. To fix the ideas, we illustrate the operation with c, the one with $\bar{c}$ being dual. At the beginning, the pattern is aligned with the text, left justified, and we compute the fusion and the values $c(1)$ by straightforward methods, in $O(m)$ time. As part of this computation, we also initialize the following items at $k = 1$.

- The number $S(k)$ of pattern runs of which the contribution to the match won't undergo a change upon unit shift.
- The number $DU(k)$ of pattern runs that will each witness a unit decrement in their contribution upon a single shift.
- The number $IU(k)$ of pattern runs that symmetrically experience unit increments upon shift.

The idea of the algorithm is that as long as there is no bump, we will have that $c(k+1) = c(k) + IU(k) - DU(k)$, whereas $IU(k+1) = IU(k)$ and $DU(k+1) = DU(k)$. Hence, within this regime the value of c can be computed in constant time per shift, whether the shift itself be unitary or multiple. In order to know the APS at every position of the text, we still need to compute the winning match $Max(c, \bar{c})$ between identity and switch. It is clear how to do that by comparing the two scores at each position of the text, at a cost of $O(n)$ operations. However, in view of the bi-modal behavior of $Max(c, \bar{c})$ in between bumps, this extra term is not needed here: just giving this value at bumps and the slope of the straight lines of c and $\bar{c}$ implicitly encodes all intermediate values.

We now look at the management of bumps. Figuratively, an external left-end delimiter of a text run enters the fusion at the right end, and exits it from the left end. During this life cycle, the run to the right of this delimiter will in general alternate between *inactive* phases, characterized by the run being entirely contained in a wider pattern run, and *active* phases, that correspond to the run being broken by a pattern run delimiter. More formally, the run with left-end delimiter L_i in fusion $\mathcal{L} = (L_1, L_2, \ldots, L_p)$ is non-active if its neighbor L_j has $j < i - 1$ and is active if $j = i - 1$ or $j = i$. In the corresponding fusion, an inactive run shows simply re-copied shift after shift, whereas an active run appears as broken, and its first and last segments undergo unit increases and decreases in size, respectively, at every shift. Note that there are only at most r_p active runs, hence r_p bumps awaiting at any given time. The corresponding sizes all vary in time, but updating them all at every step would charge an unacceptable $O(n \times r_p)$. Luckily, we do not need to, as we are only interested in the run responsible for the bump distance, and this does not change in between bumps.

Observe that, when measured relative to the origin of the pattern, the delimiters of pattern runs in the fusion do not change in time, only text run delimiters "travel backwards" inside the pattern run partition. It remains to be said how it is determined that a bump has to take place. But this is always associated with a specific alignment of the pattern relative to the text, which is computable at the time the bump is issued. More specifically, when the pattern is initially aligned with the first location of the text, and the 1-fusion is constructed, all neighbor distances are computed and sorted by a simple bucket sort into an n-length array. Besides the neighbor distances we add to this sorted array all distances between left-end delimiters (that are beyond the end of the alignment with the pattern) and the end of the pattern + 1. This sorted array associates each text element with its neighbor. Finding the next bump is equivalent to finding the minimum element in this array or linked list (which can be simply done in constant time). After a bump occurs, all text elements who have "bumped" their neighbors need to find new neighbors and to insert them into the list. This can be done straightforwardly, in time $O(\log m)$ by implementing a binary search tree over the list.

As part of the management of the bump, we also update $SU(k)$, $DU(k)$ and $IU(k)$ in an obvious way. The remaining details are easy and left for an exercise. As there are only $r_p \times r_t$ bumps, then clearly the time complexity of this algorithm is $O(r_p \times r_t \times \log m + n)$, but it is not difficult to improve it, e.g., through resort to efficient priority queue or finger tree implementations (see, e.g., [7]).

Note that there is still an $O(n)$ term charged by the auxiliary array. An alternative to the above scheme is offered by the observation that when a text run issues a bump for the first time, it is possible to predict all of its subsequent r_p bumps, and these correspond to the sequence of pattern runs, in reverse order. Assuming all bumps caused by the same text run are kept in a list, every such run will require the insertion of $r_p + 1$ items in a list of currently issued runs containing at most m elements at any given time. Each insertion can be performed in $O(r_p \ \log\log(m/r_p))$ through resort to integer finger merging techniques, leading to an $O((r_p \times r_t) \ \log\log(m/r_p))$ time algorithm that uses $O(m)$ auxiliary space.

4 When Only One of the Alphabets is Binary

We now relax the condition of binary alphabet and consider the more general case where $|\Sigma_p| = 2$ while Σ_t is arbitrary (the dual case leads to similar developments, which will be left to the reader). The structure of the fusion, particularly its restriction to a single run of the pattern facing the text, remains essentially unaffected, as seen in Figures 3 and 4.

We focus now again on a single run of the pattern (see Fig. 4) and consider the changes in its contribution to the matching under unit shifts. Let l be (the length of) this run, and $x_i \dots x_{i+l}$ the substring of text facing it, we still have the following.

Fact 2 *The update of the contribution of the run to the match depends only the values of x_i and x_{i+l}. Specifically, this contribution is unchanged if $x_i = x_{i+l}$.*

If this is not the case, that is, $x_i \neq x_{i+l}$, then the number of hits of character x_i against

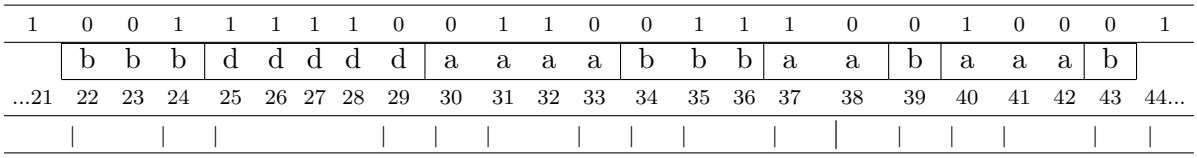

Figure 3: Fusion of pattern and text for arbitrary text

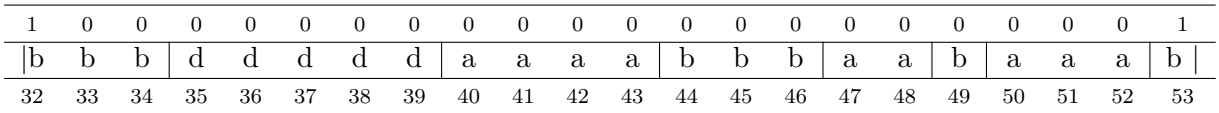

Figure 4: Section of a pattern facing one run in the text for arbitrary text alphabet

the run will decrease by 1, and the number of hits of character x_{i+l} against the run will increase by 1, while the number of hits of the other characters of Σ_t within $x_{i+1}, \ldots, x_{i+l-1}$ against the run will not change. It is convenient to single out those characters of Σ_t for which some run is currently broken by the delimiter of some pattern run, and distinguish these *crossing* characters from the rest. In Figure 4, for instance, b is a crossing character, due to the text run (32–34), and a is also a crossing character, due to the text run (50–52), while the letter d is not crossing, since none of its runs is interrupted by the pattern. Upon performing a unit shift for the pattern, b is still crossing on account of the text run (32–34), while the a in positions (50–52) loses this status. We similarly partition the set of runs of each individual character into *crossing* and *non-crossing runs*, with obvious meaning.

To keep track of the numbers of hits of the text characters against the 0's and 1's in the pattern, we use two sets of counters, one set for each character of Σ_p. Each text character has one counter which keeps track of the hits against 0's and one for the hits against 1's, but such counters are defined only for the subset of Σ_t in the active fusion. Therefore, the total number of counters at any given time is at most $2m$.

We focus on one specific text character σ and track the hits against one of the pattern characters, say 0. At the generic iteration there will be two main components to the count: the *stable* contribution from runs of non-crossing characters and the *varying* one from active runs. The latter will have in general a growing and a shrinking components, which behave symmetrically. We are interested at first in computing the hits at bump absorption. The following fact shows that knowing the current position k, the value of index h and the growing component at k_h is all is needed to compute the growing component at any future time.

Fact 3 *At step k, let $k_1, k_2, ..., k_h$ be the positions of the text at which currently open bumps were issued for the character σ. Then, the growing contribution of the hits at k obeys*

$$\begin{aligned} S(k, k_1) &= k - k_1 \\ S(k, k_h) &= S(k, k_{h-1}) + h \cdot (k - k_h). \end{aligned}$$

Proof: We have

$$S(k_h) = \sum_{g=1}^{h}(k - k_g) = \sum_{g=1}^{h-1}(k_h - k_g) + h(k - k_h) = S(k_{h-1}) + h(k - k_h)$$

∎

Consider now the sequence of all text run delimiters in the form $(i, i+1)$ such that $x_i = \sigma \neq x_{i+1}$. Every bump involving such delimiters will be called an *σ-bump*. Our observations and the above recurrence support the following

Fact 4 *For every character $\sigma \in \Sigma_t$, the hits at every σ-bump against either character of Σ_p can be computed consecutively in constant time.*

Along these lines, we can state:

Fact 5 *There is an $O(r_t \times r_p)$ algorithm to compute the hits at every σ-bump for every character $\sigma \in \Sigma_t$.*

Facts 4 and 5 convert our problem into a geometric problem: we are given a set of $O(r_t \times r_p)$ linear segments and wish to find their upper envelope. This problem is solved for k segments in time $O(k\alpha(k)\log k)$ and $O(k\alpha(k))$ storage [4], where $\alpha(k)$ is the (extremely slowly growing) inverse of the Ackermann's function. To be specific, we have two systems of linear segments, where one system lists the hits of all characters of Σ_t against character $0 \in \Sigma_p$, while the other similarly lists hits against $1 \in \Sigma_p$. At any text position, the optimum APS consists of the maximum achievable by taking the hits scored by two distinct text characters at that position. This is clearly the sum of the maxima in the two systems if the characters producing these maxima are different. If, on the other hand, these are the same text character, then we need to know the characters producing the maximum and second maximum at that position. Trying both combinations will produce the winning bijection. This can be obtained, e.g., by peeling the segments atop the upper envelope and then recomputing the latter. Note that this operation may result in a set of segments that constitutes substantial alteration of the original set, however, the number of segments at the outset is still linear in the original number.

The above arguments adapt to the case of binary Σ_t and arbitrary Σ_p. This leads to the following

Fact 6 *There is an $O(n + (r_p \times r_t)\ \alpha(r_t)\ \log(r_t))$ algorithm for computing parameterized matching with either Σ_t or Σ_p is binary and the other one is arbitrary.*

References

[1] A. Apostolico and Z. Galil (Eds.), *Pattern Matching Algorithms*, Oxford University Press, New York (1997).

[2] B. S. Baker, "Parameterized Duplication in Strings: Algorithms and an Application to Software Maintenance", *SIAM Journal of Computing*, **26**, 5, 1343-1362, (1997).

[3] B. S. Baker, "Parameterized Pattern Matching: Algorithms and Applications", *Journal Computer System Science*, **52**, 1, "28–42", (1996).

[4] H. Edelsbrunner, L.J. Guibas and M. Sharir "The Upper Envelope of Piecewise Linear Functions: Algorithms and Applications" *Discrete Computational Geometry* **4**, 311-336, 1989.

[5] M. Fischer and M. Paterson, "String Matching and Other Products", *Complexity of Computation* (R. Karp, Ed.), SIAM-AMS Proceedings, 113–125 (1974).

[6] M.L. Fredman and R.E. Tarjan, "Fibonacci Heaps and their Use in Improved Network Optimizations Algorithms", *J. ACM* **34**:3, 596-615 (1987).

[7] D.B. Johnson, "A Priority Queue in which Initialization and Queue operation take $O(\log \log D)$ Time", *Mathematical Systems Theory*, *15*, 295–309 (1982).

[8] M.Y. Kao, T.W. Lam, W.K. Sung, and H.F. Ting, "A Decomposition Theorem for Maximum Weight Bipartite Matching", *SIAM J. Computing* **31**:1, 18-26 (2001).

[9] J. Wang, B. Shapiro, and D. Shasha (Eds.), *Pattern Discovery in Biomolecular Data: Tools, Techniques, and Applications* Oxford University Press (1999).